DOCKERIZING A RAG APP

AI Engineer Ready · Week 7 — One Container, Two Processes

GOLDEN RULE Same image everywhere — only $PORT and secrets change per environment; only Streamlit's port is ever published.

1. One image, every environment

The Docker image itself never changes between local, Render, and Azure — only the *configuration around it* differs: which port the platform routes to, and where secrets come from (env vars vs. a platform's secret store). That's the entire point of reading $PORT and API keys from the environment instead of hardcoding them.

2. Building and running locally

```
docker build -t multi-document-assist .
docker run -p 8080:8080 \
-e OPENAI_API_KEY=$OPENAI_API_KEY \
-e COHERE_API_KEY=$COHERE_API_KEY \
-e OPENAI_MODEL=gpt-4o-mini \
multi-document-assist
```

Only one port is published — 8080 — because that's Streamlit's port and the only one meant to be reached from outside. FastAPI's 8000 is never published; it doesn't need to be, since nothing outside the container ever calls it directly.

3. What belongs in the Dockerfile vs. entrypoint.sh

File	Job
Dockerfile	Install Python, copy requirements.txt + src/, install deps, set CMD to entrypoint.sh
.dockerignore	Keep .venv, .env, __pycache__, data/ test files, and .git out of the image
entrypoint.sh	Runtime order: start FastAPI → poll /health → start Streamlit on $PORT
requirements.txt	Pinned versions for both the FastAPI and Streamlit process — same image, same deps

■ Pitfall — Forgetting .dockerignore

Without a .dockerignore, docker build happily copies your local .venv, .env (real API keys!), and .git history into the image layer. That bloats the image and can leak secrets into an image you push to a registry. Always exclude them explicitly.

4. Verifying a build before you ship it

- Build succeeds with no errors: docker build -t <name> .
- Container boots and FastAPI reports healthy (check container logs for the health-gate message)
- Streamlit is reachable on the mapped port and the upload → ask flow works end to end
- Re-run the exact same image with different env vars to confirm nothing is hardcoded

✔ Same image, different config

If you ever need to rebuild the image just to change an API key or a port, something is hardcoded that should be an environment variable. A production-grade image is built once and configured per-environment.

5. Interview Prep

Q: Why does this app publish only one port from the container?

A: Streamlit is the only process meant to be reached externally; FastAPI is internal-only by design, so publishing its port would just open an unintended surface for no benefit.

Q: What's the risk of skipping .dockerignore?
A: Local secrets (.env), virtual environments, and git history can get baked into the image, bloating it and potentially leaking credentials to anyone who pulls the image.

Q: Why read $PORT from the environment instead of hardcoding 8080?
A: Different hosting platforms assign different ports at runtime; reading $PORT keeps the same image deployable everywhere without a code or Dockerfile change.
